

VoteChain

Loris Marongiu & Michele Molineri

Panoramica del Sistema

VoteChain è un Proof of Concept (PoC) per un sistema di votazione elettronica decentralizzato basato su blockchain, progettato per supportare sia referendum che raccolte firme.

L'obiettivo primario è garantire la segretezza del voto, la resistenza alla manipolazione e la verificabilità pubblica del processo elettorale, sfruttando le proprietà crittografiche e di immutabilità di una rete blockchain.

Il design architetturale si ispira al meccanismo commit-reveal: ogni votante prima pubblica un impegno crittografico (commit) del proprio voto, successivamente lo rivela (reveal) durante la fase di spoglio. Questo schema separa il momento dell'espressione del voto dal momento della sua decodifica, impedendo a chiunque, incluso il governo, di conoscere le intenzioni degli elettori prima della chiusura delle urne.

Attori del Sistema

Governo (Deployer)

Il governo è l'entità che effettua il deploy del contratto sulla blockchain e ne detiene i privilegi amministrativi.

Le sue responsabilità comprendono:

- Creare e configurare nuovi referendum (titolo, opzioni di voto, durata)
- Avviare la fase di spoglio al termine del periodo di votazione
- Chiudere ufficialmente lo spoglio e calcolare i risultati

Il governo non ha accesso ai contenuti dei voti durante la fase di commit, garantendo così che il processo rimanga imparziale fino all'apertura ufficiale delle urne.

Questo account è stato pensato nella pratica come un **multisig M to N**.

Cittadino (Votante)

Nel sistema reale (produzione), il cittadino si autentica tramite SPID (Sistema Pubblico d'Identità Digitale), ottenendo un identificatore univoco federato. Nel PoC, questa fase è simulata e viene utilizzato un wallet MetaMask. Le interazioni del cittadino con il sistema comprendono:

- Generazione del wallet dedicato al referendum
- Espressione e modifica del voto (fase di commit)
- Conferma del voto durante lo spoglio (fase di reveal)

Architettura:

Generazione del Wallet per Referendum

Una delle caratteristiche innovative di VoteChain è la creazione di un wallet monouso per referendum. Aniché utilizzare un wallet generico del cittadino, viene derivato un indirizzo Ethereum specifico per ogni coppia (cittadino, referendum).

Il processo di derivazione avviene come segue:

```
seed = hash(SPID_ID, referendum_key)
```

Dove *referendum_key* è una chiave autogenerata dal contratto al momento della creazione del referendum. Da questo seed viene derivato deterministicamente un wallet Ethereum (coppia chiave privata / indirizzo pubblico).

Questo meccanismo garantisce che:

1. Lo stesso cittadino generi sempre lo stesso wallet per lo stesso referendum (riproducibilità)
2. Il wallet sia inutilizzabile in qualsiasi altro contesto (isolamento)
3. Non sia possibile correlare l'identità SPID all'indirizzo on-chain senza conoscere entrambi i segreti (anonimizzazione)
4. Un ipotetico attaccante non può ricostruire la storia delle transazioni tra i vari wallet dello stesso cittadino a meno che non possiede il suo spidID (insider dello Stato)

Purtroppo questa dinamica non è stata applicata nel PoC ed è rimasta solo concettuale per motivi strutturali: deriva direttamente dalla proprietà del modello di account di Ethereum e della scelta di mantenere l'intera logica on-chain pilotata da un frontend statico (limite di github pages) e dal wallet dell'utente (MetaMask) senza backend applicativo.

Fase 1 - Commit: Espressione del Voto

Per impedire l'analisi statistica delle transazioni on-chain, i valori che rappresentano le opzioni di voto (es. SI, NO) sono mappati a identificatori opachi generati casualmente per ogni referendum.

Ad esempio:

Opzione	Valore Opaco
SI	0x3f7a9c...
NO	0xb21e04...

Questa mappatura è nota solo al contratto, e viene rigenerata ad ogni referendum, rendendo impossibile dedurre il voto dall'osservazione della transazione.

Anche questa caratteristica che offre un livello di sicurezza maggiore non è implementabile senza backend per lo stesso motivo citato prima (i contratti sono pubblici e non posseggono segreti), quindi abbiamo deciso di hasharlo per il PoC essendo coscienti che è deterministico.

voto_opaco = hash(indirizzo, indice, label)

indirizzo = indirizzo del contratto *Referendum*

indice = posizione nell'array *optionsLabels* passato alla creazione

label = stringa corrispondente all'opzione

Struttura del Commit

Il cittadino sceglie il proprio voto e inserisce un nonce (password monouso a sua scelta). Il commit viene costruito come:

commit = hash(voto_opaco, nonce)

Questo digest viene inviato on-chain tramite una transazione dal wallet dedicato al referendum. Il contratto registra il commit senza poter risalire al voto effettivo, perché non conosce né il nonce né la mappatura invertita del voto opaco in quel momento (nel sistema pensato).

Modifica del Voto

Osservando alcuni paesi del nord che hanno digitalizzato le votazioni (es. Estonia), abbiamo appreso un pattern di sicurezza che questi applicano e l'abbiamo riproposto nel nostro progetto: il cittadino può cambiare il proprio voto prima dell'inizio della fase di spoglio. Per farlo, si ri-autentica con le stesse modalità, esprime un nuovo voto e sceglie un nuovo nonce. Prima di registrare il nuovo commit, il contratto esegue una funzione di verifica che:

1. Recupera tutte le transazioni storiche emesse dal wallet del cittadino per quel referendum
2. Per ogni transazione, calcola **hash(voto_opaco_X, nonce_proposto)** per tutte le opzioni di voto possibili X
3. Controlla che nessuno dei digest calcolati coincida con il commit della transazione esistente

Se il nuovo nonce non compare in nessuna transazione precedente, il nuovo commit viene accettato e registrato on-chain. Le transazioni di commit precedenti rimangono sulla blockchain ma diventano irrilevanti ai fini dello spoglio, poiché si tiene in conto solo l'ultimo voto.

Fase 2 - Reveal: Conferma e Spoglio

Avvio dello Spoglio

Trascorso il periodo di votazione configurato dal governo, viene invocata la funzione di apertura dello spoglio. Da questo momento i cittadini non possono più modificare il proprio voto e devono procedere alla fase di conferma.

Conferma del Voto (Reveal)

Ogni cittadino ri-accede al sistema e pubblica il proprio nonce in chiaro in una transazione di reveal. Il contratto verifica che:

hash(voto_opaco(V), nonce_rivelato) == commit_registrato

Se la verifica fallisce (nonce errato o voto dichiarato non corrispondente), il cittadino può ripetere la fase di conferma fornendo i dati corretti. Se la verifica ha successo il voto è considerato confermato. In entrambi i casi il nonce viene pubblicato in chain per la verifica.

Conteggio dei Voti

Una volta che tutti i cittadini hanno completato il reveal (o scaduto il termine), il governo invoca la funzione di chiusura dello spoglio. Il conteggio avviene tramite la stessa funzione di verifica usata in fase di commit, applicata ora in modalità di lettura aggregata:

Per ogni wallet *i* di cittadino e per ogni opzione di voto **V**:

exists transazione(t_i) t.c. hash(voto_opaco(V) , nonce_rivelato_i) = commit (t_i)

Se questa condizione è soddisfatta, il voto del cittadino *i* per l'opzione **V** viene conteggiato. Schema del Flusso Completo

Tuttavia, per prevenire lo spam di proposte infondate che invaderebbero la blockchain e l'agenda istituzionale, il sistema introduce un meccanismo di cauzione anti-spam: chiunque voglia avanzare una proposta deve bloccare una quantità di token nel contratto. Questa cauzione viene gestita automaticamente dallo smart contract in base all'esito della proposta.

Ciclo di Vita di una Proposta

1. Creazione della Proposta (Proponente)

Un cittadino autenticato via SPID genera il proprio wallet dedicato alla proposta (stesso meccanismo):

hash(SPID_ID || proposal_key)

Per avviare la raccolta firme, il proponente invia una transazione che include:

- Il *testo* della proposta
- La *descrizione* della proposta
- Un *deposito cauzionale* in token ETH, la cui soglia minima è definita dal contratto
- il *timeout* della proposta (hardcodato nel codice, uguale per tutti, una settimana)

La cauzione viene bloccata (locked) nello smart contract e non è accessibile né al proponente né al governo fino alla “risoluzione” della proposta (scadenza del timeout).

2. Firma della Proposta (Firmatari)

Ogni cittadino che vuole aderire accede con il proprio wallet dedicato alla proposta e invia una transazione di firma.

Il contratto tiene traccia di tutti i wallet firmatari e verifica che ciascun wallet abbia firmato al più una volta (unicità garantita dall’isolamento del wallet per proposta a livello concettuale, nel PoC attuale si tiene traccia con una mappa del tipo **Proposta, msg.sender -> haFirmato (bool)**).

3. Raggiungimento del Quorum

Alla scadenza del periodo di raccolta, il contratto verifica automaticamente se il numero di firme valide ha raggiunto la soglia (quorum) stabilita al momento della creazione. Esistono due esiti:

Esito	Condizione	Conseguenza sulla Cauzione
Proposta ammessa	Firme >= quorum	La cauzione viene restituita al proponente
Proposta respinta	Firme < quorum	La cauzione viene trattenuta al 50% dallo Stato e restituita al proponente per l'altro 50%

4. Valutazione Istituzionale

Se la proposta raggiunge il quorum, viene formalmente trasmessa all’istituzione (governo). L’istituzione la può approvare per trasformarla successivamente in un referendum o respingerla.

Logica Anti-Spam della Cauzione

Il meccanismo della cauzione risolve un problema classico dei sistemi di governance decentralizzata: senza un costo, chiunque potrebbe inondare il sistema con proposte prive di sostegno reale. La cauzione agisce come segnale di credibilità:

- Il proponente ha un incentivo economico a raccogliere firme reali, poiché solo raggiungendo il quorum rientra in possesso della cauzione
- I firmatari non pagano alcuna cauzione, abbassando la barriera alla partecipazione democratica
- Il costo di una proposta fallita è interamente a carico del proponente, disincentivando lo spam senza penalizzare i cittadini sostenitori

Il motivo del timeout

Il meccanismo del timeout serve esclusivamente come garanzia che i token non rimangano bloccati per sempre dal sistema.

Precedentemente avevamo pensato ad un'altra soluzione: un ROI per chi crea petizioni che superano il minimo di firme, plusvalore creato dalle stesse petizioni che non raggiungono il minimo di firme necessarie. Questo meccanismo avrebbe sicuramente favorito la votazione delle persone dal momento che crea opportunità di guadagno, però è stato deprecato per due motivi principali:

- **Motivo numero 1:** Questa dinamica non rappresentava la realtà, nel mondo reale non arrivano così tante proposte da creare del guadagno
- **Motivo numero 2:** Per creare questa dinamica avevamo bisogno di raggruppare i periodi di votazione in cicli controllati dall'ente governativo per la chiusura della validità e la raccolta dei fondi al fine della spartizione